



**Universidad de las Fuerzas Armadas ESPE**

**Departamento: Ciencias de la computación**

**Carrera : Ingeniería de software**

**Taller N : 4**

---

## **1. Información General**

- **Asignatura:** Análisis y diseño de Software
- **Apellidos y nombres de los estudiantes:**
  - ✓ Guevara Haro, Ariel Dunkeny
  - ✓ Reinoso Torres, Dylan Gabriel
  - ✓ Salcedo Chichande, Micaela Stefania
- **NRC:** 22426
- **Fecha de realización:** 10/07/2025

---

## **2. Objetivo del Taller y Desarrollo**

### **Objetivo del Taller:**

Aplicar el Principio de Responsabilidad Única (SRP) en el desarrollo de un sistema CRUD para estudiantes, con el fin de fortalecer el diseño modular y mantenible del software, promoviendo buenas prácticas de programación orientada a objetos y facilitando la escalabilidad del sistema.

## **1. Introducción**

El presente taller tiene como propósito aplicar conceptos fundamentales del diseño de software, en particular el Principio de Responsabilidad Única (SRP), uno de los pilares de los principios SOLID. A través del desarrollo de un sistema CRUD para la gestión de estudiantes, se busca reforzar la importancia de dividir las responsabilidades en clases separadas, lo cual mejora la mantenibilidad, flexibilidad y escalabilidad del código. Este ejercicio práctico permite a los estudiantes comprender cómo una buena arquitectura basada en SRP facilita el desarrollo modular, la identificación clara de responsabilidades y la evolución del sistema sin afectar componentes no relacionados.

## **2. Desarrollo**

### **2.1. ¿Qué es SRP?**

**SRP significa Single Responsibility Principle (Principio de Responsabilidad Única) que establece**

El SRP (Single Responsibility Principle) es un principio del diseño de software que dice que cada clase debe tener una única razón para cambiar, es decir, solo debe encargarse de una responsabilidad específica. Esto ayuda a que el código sea más limpio y fácil de mantener. Según Robert C. Martin, uno de los creadores de los principios SOLID, “una clase debe tener una, y solo una, razón para cambiar” (Martin, 2003)

## 2.2. *Ventajas de aplicar SRP*

- Si cambias cómo se imprimen los datos, no tocas la clase **Estudiante**, porque eso ahora está en la clase EstudiantePrinter.
- Si cambias cómo se guardan, tampoco tocas el **modelo**, ya que el guardado está en la clase EstudianteRepository.
- Cada clase tiene UNA SOLA razón para cambiar lo que hace el código más limpio, fácil de mantener y menos propenso a errores cuando se hace una modificación.

## 2.3. *Ejemplo que no cumple SRP*

Una mala aplicación del Principio de Responsabilidad Única (SRP) ocurre cuando una clase asume múltiples responsabilidades, mezclando lógica de negocio, presentación y acceso a datos en un solo lugar. Este tipo de diseño hace que el código sea difícil de mantener, probar y escalar.

### 2.3.1. Código

```
public class Estudiante {  
  
    private int id;  
  
    private String nombres;  
  
    private String apellidos;  
  
    private int edad;  
  
  
    // Lógica de negocio  
  
    public boolean esMayorDeEdad() {  
  
        return edad >= 18;  
  
    }  
}
```



```
}
```

```
// Presentación
```

```
public void imprimirDatos() {
```

```
    System.out.println("ID: " + id);
```

```
    System.out.println("Nombre: " + nombres + " " + apellidos);
```

```
    System.out.println("Edad: " + edad);
```

```
}
```

```
// Acceso a datos
```

```
public void guardarEnBaseDeDatos() {
```

```
    // Código que realiza la conexión a la base de datos
```

```
    // y guarda el estudiante
```

```
}
```

```
}
```

### 2.3.2. Problemas encontrados

- Si cambia la forma de imprimir los datos, hay que modificar esta clase.
- Si cambia la lógica para guardar en la base de datos, también hay que modificar esta clase.
- Si se requiere cambiar la lógica de negocio (por ejemplo, una nueva regla para determinar la mayoría de edad), también se modifica aquí.



### 3. Conclusiones

- El uso correcto del principio de responsabilidad única (SRP) en la arquitectura del proyecto CRUD proporciona una estructura altamente mantenible y comprensible. Cada clase está claramente definida para encargarse de una sola tarea específica, lo que significa que, si se requiere realizar un cambio en la forma de presentación, en la lógica de negocio o en el acceso a los datos, este se puede llevar a cabo sin afectar otras partes del sistema. Este aislamiento de responsabilidades reduce significativamente el riesgo de introducir errores no intencionados en el código y facilita el trabajo en equipo, ya que varios desarrolladores pueden trabajar simultáneamente en diferentes capas del sistema sin generar conflictos. En un contexto académico y profesional, esto demuestra la madurez del diseño y la adherencia a buenas prácticas de ingeniería de software. A largo plazo, esta forma de estructurar el código favorece la escalabilidad, ya que permite extender o modificar el sistema de forma modular y ordenada.
- La arquitectura basada en SRP permite una clara asignación de responsabilidades dentro del sistema, lo que facilita tanto el desarrollo como la comprensión del código. En el ejemplo analizado, cada clase cumple una función específica: el modelo representa datos, el DAO gestiona el acceso a los mismos, el controlador coordina las operaciones, y la interfaz de usuario interactúa con el usuario final. Esta división no solo es útil para mantener el código limpio, sino también para reforzar la lógica detrás del diseño orientado a objetos, donde cada objeto tiene un propósito bien definido. Esta claridad se traduce en beneficios prácticos al momento de realizar pruebas, ya que cada componente puede probarse de forma aislada. Además, facilita la implementación de principios como

el test-driven development (TDD), ya que al reducir las dependencias internas, se vuelve más sencillo crear pruebas unitarias. Esta forma de estructuración mejora también la documentación del sistema, ya que cada parte se describe con precisión y sin ambigüedades.

- La correcta aplicación de la arquitectura basada en SRP no solo mejora la calidad del código, sino que también promueve su reutilización en diferentes contextos. Por ejemplo, la clase EstudianteDAO puede adaptarse fácilmente para usarse con otro tipo de entidades simplemente modificando su implementación, sin afectar las clases Estudiante o EstudianteUI. Del mismo modo, si se decide cambiar la forma de mostrar la información (por ejemplo, de consola a una interfaz gráfica o a una aplicación web), esto puede hacerse únicamente modificando la clase de presentación, sin alterar el modelo ni la lógica de persistencia. Esta capacidad de adaptación es fundamental en proyectos de mediano a largo plazo, donde los requerimientos del sistema pueden cambiar debido a necesidades del cliente o avances tecnológicos. En este sentido, el SRP se convierte en una herramienta poderosa para construir sistemas flexibles y preparados para el cambio, permitiendo a los desarrolladores responder rápidamente ante nuevas exigencias sin tener que rehacer el sistema completo.

## Referencia

Martin, R. C. (2003). *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall.